



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE SOBRAL

Voltímetro Digital de Duas Escalas com o microcontrolador PIC18F4520

Luis Felipe Pessoa Lacerda
engluisfelipecessoa@gmail.com

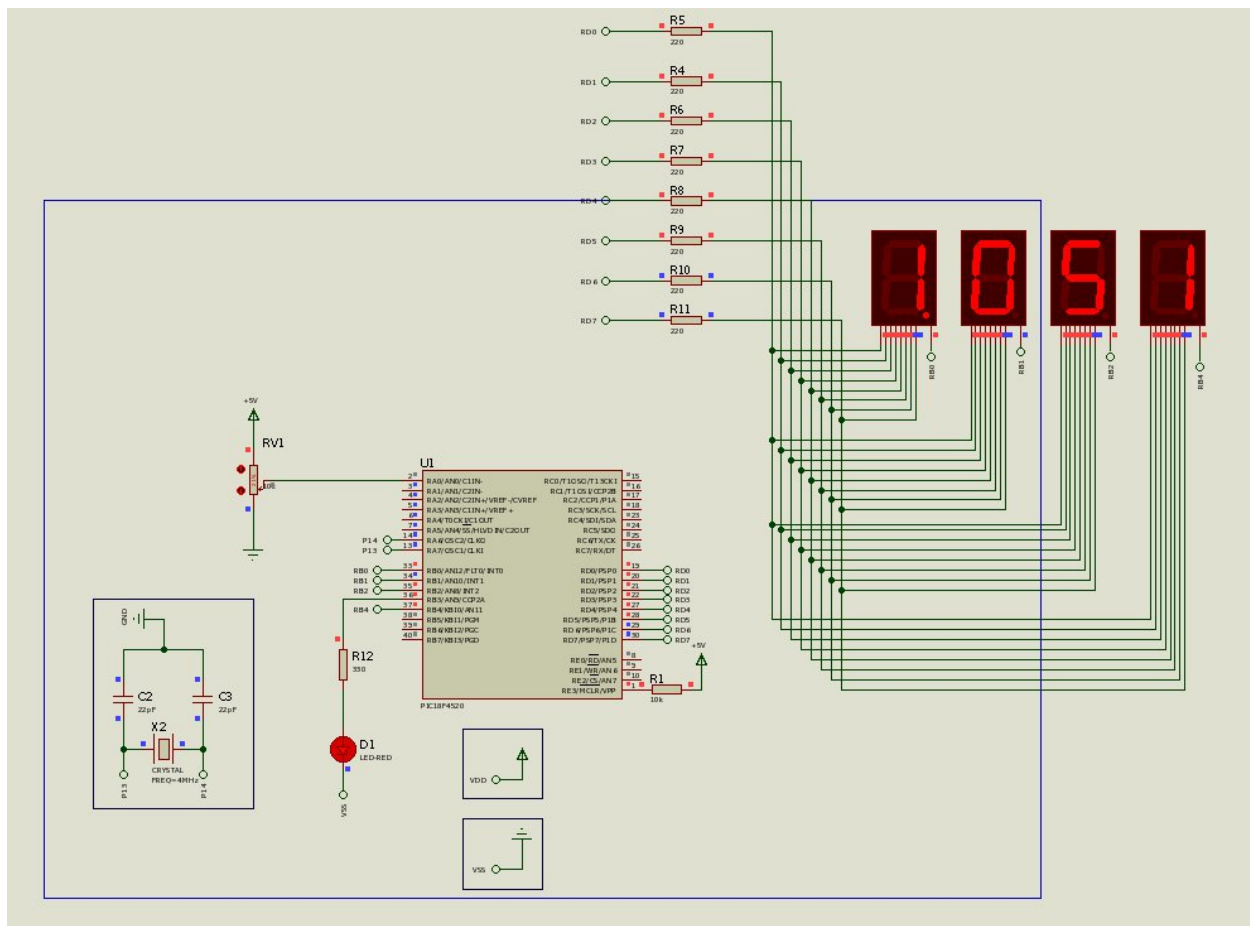
1. Introdução e Objetivo

Este relatório documenta o desenvolvimento de um voltímetro digital de duas escalas, implementado com o microcontrolador PIC18F4520. O sistema mede uma tensão analógica de entrada (0 a 5 V) e a exibe em quatro displays de 7 segmentos, alternando automaticamente entre duas formas de apresentação: Escala 1 (0 a 999 mV): tensões de entrada abaixo de 1 V; Escala 2 (1 V a 5 V): tensões de entrada de 1 V até o fundo de escala de 5 V.

A troca de escala é feita automaticamente pelo firmware, sem qualquer intervenção do usuário, e é sinalizada por um LED indicador. O formato de exibição muda conforme a escala ativa: na Escala 2, o valor é apresentado como “d0.d1d2d3” V (ex: 2.340); na Escala 1, o valor é reapresentado em mV no formato “d0d1d2.d3” mV (ex: 945.0), com o ponto decimal deslocado para a terceira posição. O uso de quatro displays foi necessário para suportar ambos os formatos.

2. Hardware Desenvolvido

2.1 Circuito no Protheus



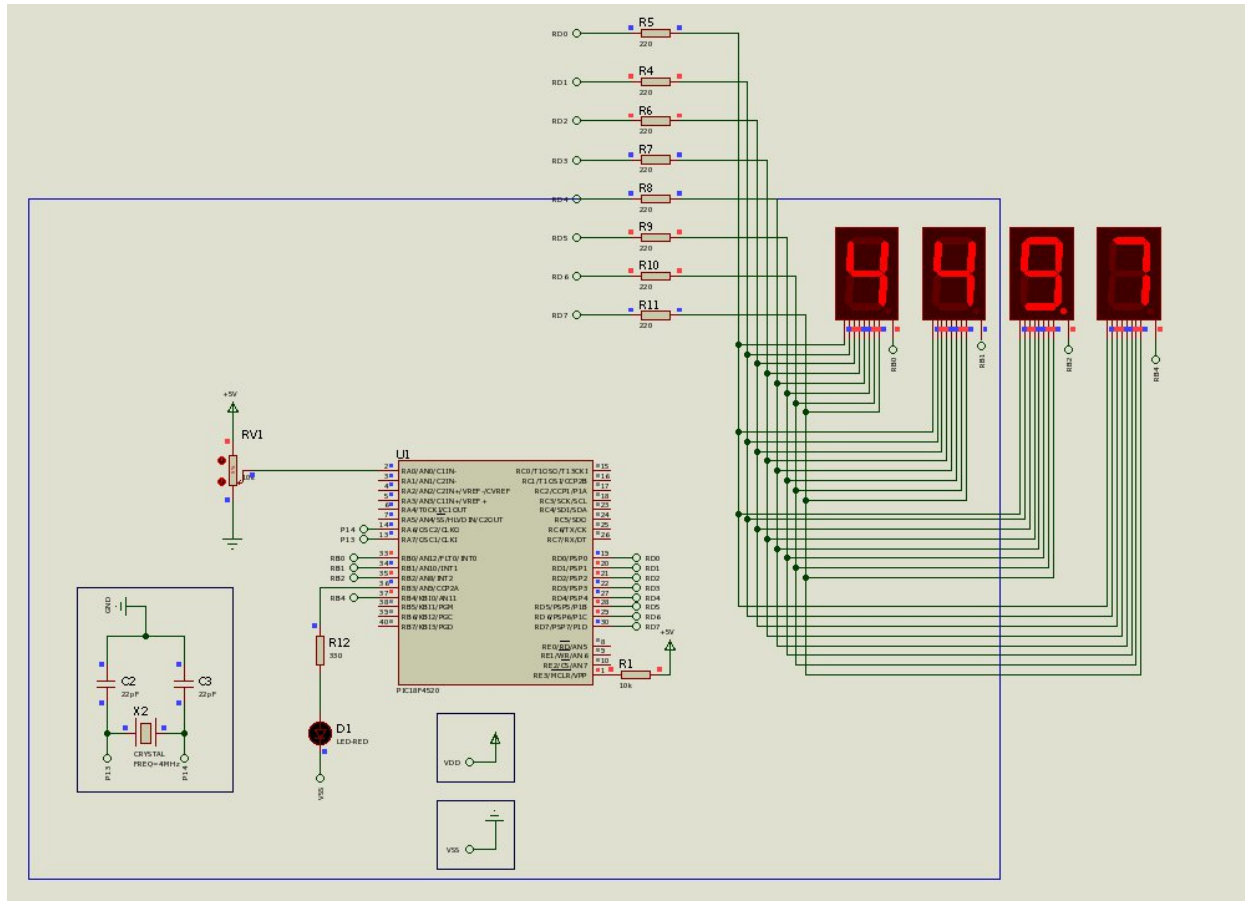


Figura 2: Exemplo para 449.7mV(< 1V)

2.2 Visão geral

O circuito é baseado no microcontrolador PIC18F4520, operando com cristal externo de 4 MHz (modo de oscilador HS). A entrada analógica é aplicada ao canal AN0 (RA0) por meio de um potenciômetro de 10 k Ω (RV1), que funciona como fonte de tensão variável de teste entre 0 e 5 V (V_{dd}), simulando o sinal que se desejasse medir. A saída é apresentada em quatro displays de 7 segmentos de catodo comum (CC), multiplexados no tempo, e um LED indica a escala ativa.

2.3 Microcontrolador e oscilador

O PIC18F4520 foi configurado em modo HS (High Speed) com cristal de 4 MHz e capacitores de 22 pF (C2 e C3).

2.3 Entrada analógica

A entrada AN0 (RA0) foi configurada como entrada analógica de 10 bits, com tensão de referência igual à própria alimentação ($V_{ref+} = V_{dd} = 5\text{ V}$, $V_{ref-} = V_{ss} = 0\text{ V}$).

2.4 Displays de 7 segmentos e seleção de dígito

Os quatro displays são do tipo catodo comum (CC). Os 8 segmentos (a-g e o ponto decimal — dp) de todos os displays são ligados em paralelo aos mesmos 8 resistores de 220 Ω (R4 a R11), acionados pelo PORTD (RD0 a RD7) do PIC.

2.5 LED indicador de escala

O LED D1 (com resistor limitador R12 de 330 Ω) foi conectado ao pino RB3. LED apagado = Escala 1 (mV); LED aceso = Escala 2 (V).

2.6 Tabela de pinagem

Pino do PIC18F4520	Função no projeto
RA0 / AN0	Entrada analógica (tensão a medir, vinda do potenciômetro RV1)
RD0 – RD7	Segmentos dos displays (a, b, c, d, e, f, g, dp), via R4–R11 (220 Ω)
RB0	Seleção do dígito 0 (dígito inteiro) — ativo em nível baixo
RB1	Seleção do dígito 1 (1ª casa decimal) — ativo em nível baixo
RB2	Seleção do dígito 2 (2ª casa decimal) — ativo em nível baixo
RB3	LED D1: indicador de escala (apagado = mV, aceso = V)
RB4	Seleção do dígito 3 (3ª casa decimal) — ativo em nível baixo
OSC1 / OSC2	Cristal de 4 MHz (X2) + capacitores C2 e C3 de 22 pF
MCLR	Pull-up de 10 k Ω (R1) para Vdd

3. Software Desenvolvido

O firmware foi escrito em C, no MPLAB X IDE com o compilador XC8, estruturado em duas partes concorrentes: o laço principal (que lê o ADC, calcula a média de 8 amostras e chama AtualizaDisplay) e a ISR do Timer0 (que a cada 2 ms apaga os dígitos, coloca o padrão certo no PORTD e acende o dígito da vez).

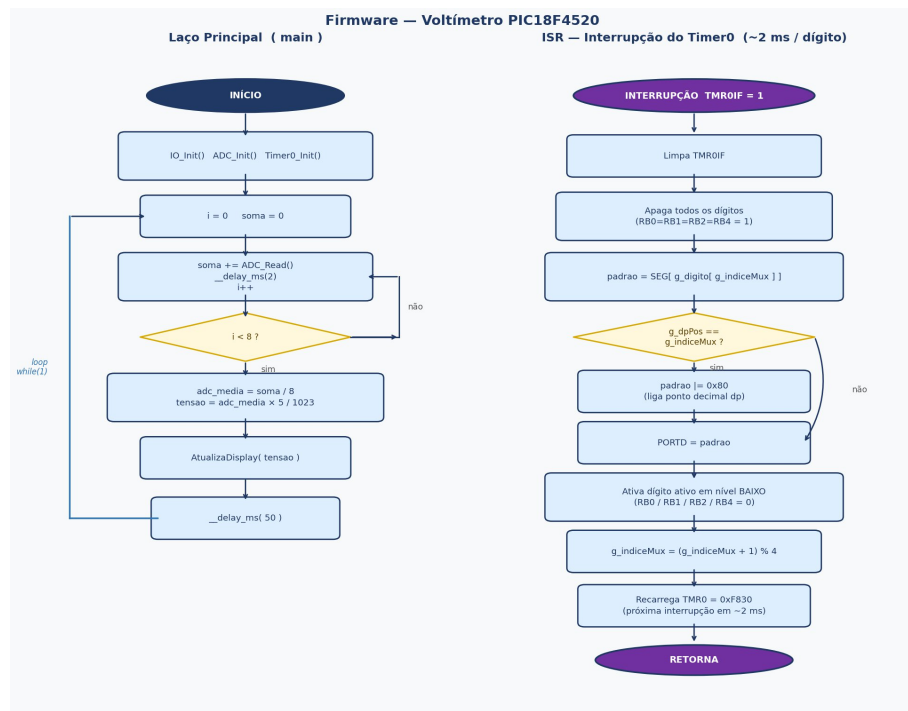


Figura 3 — Fluxograma simplificado do laço principal e da rotina de interrupção do Timer0.

3.1 Código

```

/* =====
* Voltímetro Digital de Duas Escalas - PIC18F4520
* Segunda Atividade Avaliativa - Proposta 2
*
* Com 4 DISPLAYS, para exibir 3 casas decimais nas duas escalas:
* Escala 1: 0 a 999 mV -> exibido como "XXX.X" V (1 casa decimal)
* Escala 2: 1 V a 5 V -> exibido como "X.XXX" V (3 casas decimais)
* A troca de escala é automática, baseada no valor medido (define apenas
* o dígito inteiro d0 e aciona o LED indicador).
*
* Hardware:
* - RA0/AN0 -> entrada analógica (tensão a medir), Vref = Vdd = 5V
* - RD0..RD7 -> segmentos dos displays (a,b,c,d,e,f,g,dp) via
* resistores de 220 ohm (mesma fiação em paralelo
* para os 4 displays)
* - RB0,RB1,RB2,RB4 -> seleção do dígito (displays catodo comum - CC),
* ATIVO EM NÍVEL BAIXO
* RB0 = dígito 0 (inteiro), RB1 = dígito 1 (decimo),
* RB2 = dígito 2 (centesimo), RB4 = dígito 3 (millesimo)
* - RB3 -> LED D1 indicador de escala: APAGADO = medindo em mV
* (Escala 1), LIGADO = medindo em V (Escala 2)
* - Cristal X2 = 4 MHz, C2 e C3 = 22 pF -> oscilador HS
*
* ===== */

#include <xc.h>

#pragma config OSC = HS
#pragma config WDT = OFF
#pragma config LVP = OFF
#pragma config PWRT = OFF
#pragma config BOREN = OFF
#pragma config DEBUG = OFF
#pragma config MCLRE = ON
#pragma config PBAEN = OFF

#define _XTAL_FREQ 4000000UL /* cristal de 4 MHz, conforme X2 */

/* -----
* Tabela de segmentos para display catodo comum (1 = segmento ligado)
* bit0=a bit1=b bit2=c bit3=d bit4=e bit5=f bit6=g bit7=dp
* ----- */
const unsigned char SEG[10] = {
    0x3F, /* 0 */
    0x06, /* 1 */
    0x5B, /* 2 */
    0x4F, /* 3 */
    0x66, /* 4 */
    0x6D, /* 5 */
    0x7D, /* 6 */
    0x07, /* 7 */
    0x7F, /* 8 */
    0x6F /* 9 */
};

/* Variáveis globais usadas pela rotina de multiplexação (ISR do Timer0) */
volatile unsigned char g_digito[4] = {0, 0, 0, 0}; /* [0]=inteiro, [1..3]=decimais */
volatile signed char g_dpPos = 0; /* posição do ponto decimal
(sempré após d0) */
volatile unsigned char g_indiceMux = 0;

/* ----- Protótipos ----- */
void IO_Init(void);
void ADC_Init(void);
unsigned int ADC_Read(void);
    
```

```

void Timer0_Init(void);
void AtualizaDisplay(float tensao);

/* ----- Interrupcao do Timer0 (refresh dos displays) --- */
void __interrupt() ISR_TMR0(void)
{
    if (INTCONbits.TMR0IF)
    {
        INTCONbits.TMR0IF = 0;

        /* desliga todos os digitos antes de trocar o padrao (evita "ghosting") */
        PORTBbits.RB0 = 1;
        PORTBbits.RB1 = 1;
        PORTBbits.RB2 = 1;
        PORTBbits.RB4 = 1;

        /* monta o padrao do digito atual */
        unsigned char padrao = SEG[g_digito[g_indiceMux]];
        if (g_dpPos == g_indiceMux) padrao |= 0x80; /* liga o ponto decimal */

        PORTD = padrao;

        /* habilita (nivel baixo) apenas o digito atual - catodo comum */
        switch (g_indiceMux)
        {
            case 0: PORTBbits.RB0 = 0; break;
            case 1: PORTBbits.RB1 = 0; break;
            case 2: PORTBbits.RB2 = 0; break;
            case 3: PORTBbits.RB4 = 0; break;
        }

        g_indiceMux++;
        if (g_indiceMux > 3) g_indiceMux = 0;

        /* recarrega Timer0 para ~2 ms (refresh ~125 Hz / 4 digitos -> sem flicker)
        */
        TMR0H = 0xF8;
        TMR0L = 0x30;
    }
}

void Timer0_Init(void)
{
    T0CON = 0b10001000; /* TMR0ON=1, modo 16 bits, prescaler 1:1, clock=Fosc/4=1MHz
    */
    TMR0H = 0xF8;
    TMR0L = 0x30; /* 65536-2000 = 63536 = 0xF830 -> ~2 ms por digito */
    INTCONbits.TMR0IF = 0;
    INTCONbits.TMR0IE = 1;
    RCONbits.IPEN = 0; /* sem niveis de prioridade de interrupcao */
    INTCONbits.GIE = 1;
}

void IO_Init(void)
{
    TRISD = 0x00; /* PORTD = segmentos (saida) */
    PORTD = 0x00;

    TRISB &= 0xE0; /* RB0,RB1,RB2,RB3,RB4 como saida (3 sel. digito + LED + 1
    sel. digito) */
    PORTB |= 0x07; /* digitos 0,1,2 desligados (nivel alto = apagado, catodo
    comum) */
    PORTBbits.RB4 = 1; /* digito 3 desligado tambem */
    PORTBbits.RB3 = 0; /* LED D1 apagado no inicio (parte de mV) */

    TRISA = 0x01; /* RA0 = entrada (AN0) */
}
    
```

```

void ADC_Init(void)
{
    ADCON1 = 0x0E; /* VCFG=00 (Vref+ = Vdd, Vref- = Vss); PCFG=1110 -> so AN0 e
analogico */
    ADCON2 = 0x91; /* ADFM=1 (result. justif. a direita), ACQT=010 (4 Tad),
ADCS=001 (Fosc/8) */
    ADCON0 = 0x01; /* CHS=0000 (AN0), ADON=1 */
}

unsigned int ADC_Read(void)
{
    ADCON0bits.GO = 1;
    while (ADCON0bits.GO_NOT_DONE);
    return ((unsigned int)ADRESH << 8) | ADRESL;
}

/* Converte a tensao medida (volts) para os 4 digitos a exhibir, aplicando
* a troca automatica de escala:
*
* Escala 2 (tensao >= 1 V): formato "d0.d1d2d3" V (ex: 2.340)
*   dpPos = 0 -> ponto apos d0
*   d0 = parte inteira (1..5)
*   d1..d3 = casas decimais em volts
*
* Escala 1 (tensao < 1 V): formato "d0d1d2.d3" mV (ex: 945.0)
*   dpPos = 2 -> ponto apos d2, exibindo XYZ.W mV
*   d0..d2 = centenas/dezenas/unidades de mV (0..999)
*   d3 = decimo de mV (sempre 0, pois resolucao do ADC ~4,89 mV)
*/
void AtualizaDisplay(float tensao)
{
    if (tensao < 0.0f) tensao = 0.0f;
    if (tensao > 5.0f) tensao = 5.0f;

    unsigned char d0, d1, d2, d3;
    signed char dpPos;

    if (tensao >= 1.0f)
    {
        /* --- Escala 2: X.XXX V --- */
        PORTBbits.RB3 = 1; /* LED aceso = escala V */

        unsigned int milisimos = (unsigned int)(tensao * 1000.0f + 0.5f);
        if (milisimos > 5000) milisimos = 5000;

        d0 = milisimos / 1000; /* parte inteira: 1..5 */
        unsigned int resto = milisimos % 1000;
        d1 = resto / 100; /* 1a decimal em V */
        d2 = (resto / 10) % 10; /* 2a decimal em V */
        d3 = resto % 10; /* 3a decimal em V */
        dpPos = 0; /* ponto apos d0: "d0.d1d2d3" */
    }
    else
    {
        /* --- Escala 1: XXX.X mV --- */
        PORTBbits.RB3 = 0; /* LED apagado = escala mV */

        unsigned int decimos_mV = (unsigned int)(tensao * 10000.0f + 0.5f);
        if (decimos_mV > 9999) decimos_mV = 9999;

        d0 = (decimos_mV / 1000) % 10; /* centenas de mV (0..9) */
        d1 = (decimos_mV / 100) % 10; /* dezenas de mV (0..9) */
        d2 = (decimos_mV / 10) % 10; /* unidades de mV (0..9) */
        d3 = decimos_mV % 10; /* decimos de mV (0..9) */
        dpPos = 2; /* ponto apos d2: "d0d1d2.d3" */
    }

    /* atualizacao "atomica" das variaveis usadas pela ISR */
}

```

```

    INTCONbits.GIE = 0;
    g_digito[0] = d0;
    g_digito[1] = d1;
    g_digito[2] = d2;
    g_digito[3] = d3;
    g_dpPos      = dpPos;
    INTCONbits.GIE = 1;
}

void main(void)
{
    IO_Init();
    ADC_Init();
    Timer0_Init();

    while (1)
    {
        /* media de 8 leituras para reduzir ruido na medida */
        unsigned long soma = 0;
        unsigned char i;
        for (i = 0; i < 8; i++)
        {
            soma += ADC_Read();
            __delay_ms(2);
        }
        unsigned int adc_media = (unsigned int)(soma / 8);

        /* converte contagem ADC para tensao: Vref=5V, resolucao 10 bits */
        float tensao = ((float)adc_media * 5.0f) / 1023.0f;

        AtualizaDisplay(tensao);

        __delay_ms(50); /* taxa de atualizacao da medida exibida */
    }
}

```

3.2 Inicialização

- IO_Init(): configura RD0–RD7 como saída (segmentos); RB0, RB1, RB2, RB3 e RB4 como saída; RA0 como entrada analógica.
- ADC_Init(): $V_{ref} = V_{dd}/V_{ss}$, canal AN0 analógico, resultado justificado à direita, 4 T_{ad} de aquisição, clock $F_{osc}/8$.
- Timer0_Init(): modo 16 bits, sem prescaler, interrupção a cada ~2 ms.

3.3 Leitura do ADC e média de amostras

A cada ciclo, o firmware realiza 8 leituras consecutivas do canal AN0, com intervalo de 2 ms entre elas, e calcula a média aritmética para filtragem de ruído.

3.4 Conversão para tensão e cálculo dos dígitos

$V_{in} = (adc_média \times 5) / 1023$. A função AtualizaDisplay() aplica lógicas distintas conforme a escala:

- Escala 2 ($V_{in} \geq 1$ V): calcula milésimos de volt ($V_{in} \times 1000$), distribui nos dígitos d0–d3 no formato “d0.d1d2d3” V, e define g_dpPos = 0 (ponto após d0). Ex: 2,34 V → 2.340.
- Escala 1 ($V_{in} < 1$ V): calcula décimos de mV ($V_{in} \times 10000$), distribui nos dígitos d0–d3 como centenas/dezenas/unidades/décimos de mV, e define g_dpPos = 2 (ponto após d2). Ex: 0,945 V → 945.0 mV.

A variável `g_dpPos` é lida pela ISR do Timer0, que acende o ponto decimal (bit 7 do PORTD) apenas no dígito cujo índice coincide com `g_dpPos` — permitindo que a posição do ponto mude dinamicamente entre escalas sem nenhuma alteração no hardware.

3.5 Multiplexação dos displays (interrupção do Timer0)

A cada interrupção (~2 ms): apaga todos os dígitos, monta padrão de segmentos no PORTD, ativa o pino de seleção do dígito da vez e avança o índice.

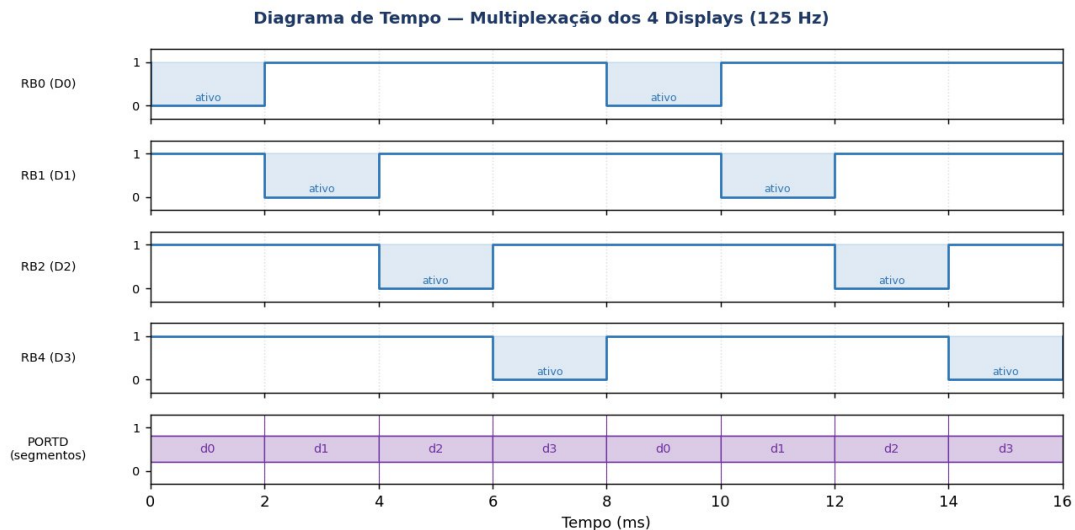


Figura 4 — Diagrama de tempo da multiplexação dos 4 displays (nível baixo = dígito aceso).

3.6 Atualização atômica dos dígitos

A função `AtualizaDisplay()` desabilita brevemente as interrupções globais (`GIE = 0`) durante a atualização dos vetores `g_digito[]` e `g_dpPos`, evitando leituras inconsistentes pela ISR.

4. Decisões e Escolhas de Projeto

4.1 Uso de 4 displays e ponto decimal dinâmico

O esquemático original previa 3 displays. Com 3 dígitos, seria impossível exibir ao mesmo tempo a parte inteira e 3 casas decimais na Escala 2 (ex: 2.340 requer 4 dígitos). O 4º display (catodo comum em RB4) resolve isso e ainda viabiliza o formato mV da Escala 1 (ex: 945.0), que também ocupa 4 posições. A posição do ponto decimal não é fixa no hardware — ela é controlada pela variável `g_dpPos`, escrita pelo laço principal (0 na Escala 2, 2 na Escala 1) e lida pela ISR, que acende o segmento `dp` somente no dígito correspondente. Assim, o mesmo hardware de 4 displays suporta os dois formatos sem qualquer mudança de circuito.

4.2 Displays catodo comum com seleção ativa em nível baixo

Pino de seleção em nível baixo durante o intervalo ativo garante que apenas um display fique aceso por vez, evitando ghosting.

4.3 Refresh via interrupção, não por delay no laço principal

A multiplexação na ISR do Timer0 mantém brilho estável independentemente do tempo gasto no laço principal, evitando flicker perceptível.

4.4 Média de 8 amostras do ADC

8 leituras separadas por 2 ms reduzem ruído sem comprometer o tempo de resposta (ciclo de 16 ms, imperceptível ao usuário).

4.5 Parâmetros do ADC (ACQT e ADCS)

Tempo de aquisição de $4 T_{ad}$ e clock $F_{osc}/8$, resultando em $T_{ad} = 2 \mu s$ — dentro do limite mínimo do datasheet ($T_{ad} \geq 1,6 \mu s$).

4.6 Manutenção do potenciômetro como fonte de teste

O potenciômetro RV1 representa de forma realista uma entrada analógica variável.

5. Memorial de Cálculos

5.1 Resolução do conversor A/D

Com $V_{ref} = 5 V$ e 10 bits (1024 níveis):

$$Resolução = V_{ref} / 1023 = 5 V / 1023 \approx 4,89 \text{ mV por LSB}$$

O último dígito exibido tem incerteza de aproximadamente $\pm 2,44 \text{ mV}$ (meio LSB).

A Figura 5 mostra a relação entre a tensão real de entrada (V_{in}) e o valor que o display exibe. O gráfico principal cobre 0–5 V, e o zoom no lado direito revela os degraus de $\sim 4,89 \text{ mV}$ — que é a resolução real do ADC de 10 bits. Serve para mostrar que o voltímetro não é contínuo, ele "pula" de valor em valor.

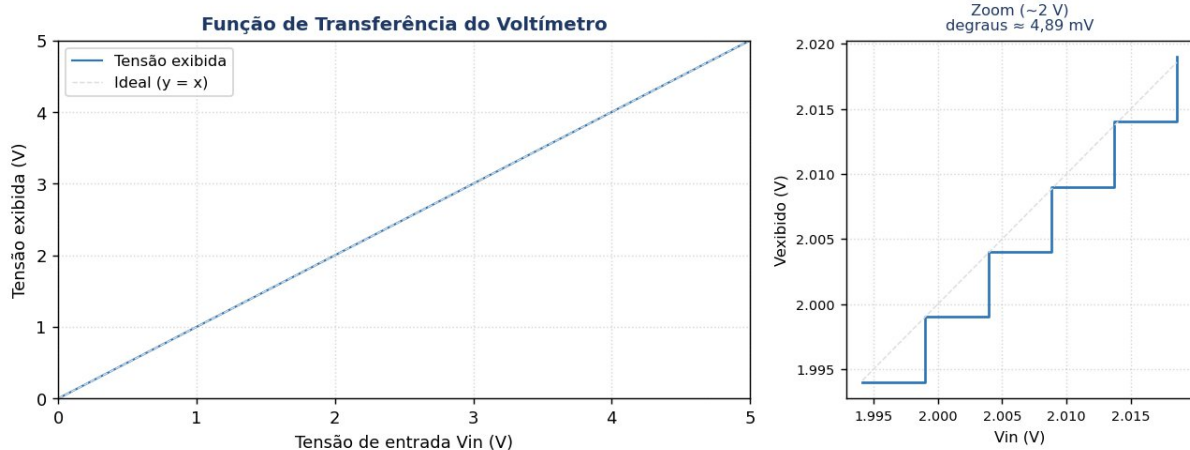


Figura 5 — Função de transferência do voltímetro. O zoom mostra degraus reais de $\sim 4,89 \text{ mV}$.

A Figura 6 mostra o erro entre a tensão real e a exibida ao longo de toda a faixa. A curva fica sempre entre as linhas vermelhas de $\pm 2,44 \text{ mV}$ (meio LSB), comprovando que o pior caso de erro é exatamente metade da resolução — comportamento esperado e correto para um ADC.

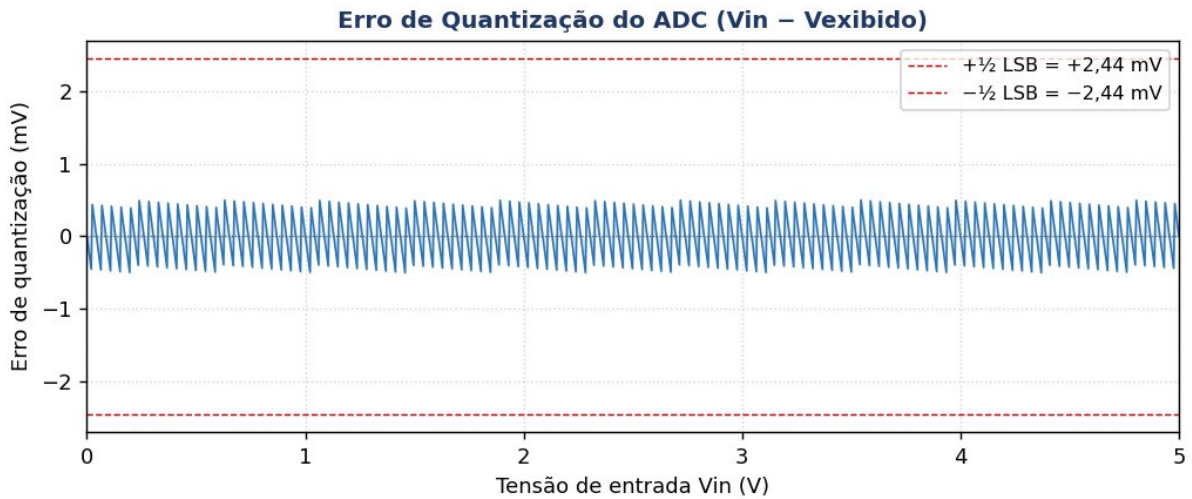


Figura 6 — Erro de quantização ($V_{in} - V_{exibido}$), limitado a $\pm 2,44$ mV.

5.2 Tempo de aquisição e conversão do ADC

$$T_{ad} = 8 / F_{osc} = 8 / 4.000.000 \text{ Hz} = 2 \mu s$$

$$T_{conversão} \approx 11 \times 2 \mu s = 22 \mu s$$

$$T_{amostra completa} \approx 8 \mu s + 22 \mu s = 30 \mu s$$

5.3 Temporização do Timer0 e taxa de refresh

$$Recarga = 65536 - 2000 = 63536 = 0xF830 \text{ (TMR0H} = 0xF8, \text{TMR0L} = 0x30)$$

$$T_{ciclo} = 4 \times 2 \text{ ms} = 8 \text{ ms} \rightarrow F_{refresh} = 125 \text{ Hz}$$

Frequência bem acima do limiar de flicker ($\sim 50\text{--}60$ Hz), com duty cycle de 25% por dígito.

5.4 Tempo de ciclo de atualização da medida

$$T_{ciclo_main} \approx 8 \times 2 \text{ ms} + 50 \text{ ms} = 66 \text{ ms} \rightarrow \sim 15 \text{ atualizações/segundo}$$

6. Casos de Teste

Ajuste o potenciômetro RV1 (ou use uma fonte DC no Proteus) para cada valor de tensão abaixo e registre o resultado observado nos displays.

V _{in} aplicada	Esperado (display)	LED esperado	Observado no Proteus
0,000 V	000.0 mV	Apagado	000,0
0,045 V	045.0 mV	Apagado	045.0
0,500 V	500.0 mV	Apagado	500.0
0,945 V	945.0 mV	Apagado	945.0
0,999 V	999.0 mV	Apagado	999.0
1,000 V	1.000 V	Aceso	1.000
1,001 V	1.001 V	Aceso	1.001

2,340 V	2.340 V	Aceso	2.340
3,275 V	3.275 V	Aceso	3.275
4,999 V	4.999 V	Aceso	4.999
5,000 V	5.000 V	Aceso	5.000

7. Conclusão

O voltímetro digital de duas escalas mede tensões de 0 a 5 V, troca automaticamente entre representação em milivolts e volts, e exibe o resultado com três casas decimais em ambas as escalas. A arquitetura de software com multiplexação na ISR garante exibição estável. A principal limitação é a resolução do ADC (~4,89 mV), que é o fator dominante de incerteza no último dígito exibido.

8. Repositório do Projeto

O código-fonte completo, bem como os arquivos do projeto e documentação complementar, estão disponíveis no repositório GitHub:

<https://github.com/engfelipelacerda/digital-voltmeter-pic18f4520.git>